

Stress-Testing n-step Soft Actor-Critic under Limited Compute

Final RL Project Report

Radmir I.

2026

Contents

1 Abstract	1
2 Introduction	2
3 Research question	2
4 Literature context	2
5 Method	3
6 Experimental setup	3
7 Results	3
7.1 Pendulum-v1 sanity check	3
7.2 HalfCheetah-v5 main ablation	4
8 Interpretation	4
9 Limitations	5
10 Future work	5
11 Conclusion	6

Short version. This project tests one central mechanism behind recent n-step SAC work: whether replacing the standard 1-step critic target with an n-step target helps Soft Actor-Critic under a limited compute budget. The main result on HalfCheetah-v5: $n=3$ improved learning compared with the 1-step baseline, while $n=5$ failed under the same hyperparameters. The main conclusion is that the n-step horizon is a stability parameter, not a simple bigger-is-better knob.

1 Abstract

This project studies a compact n-step modification of Soft Actor-Critic (SAC) under a limited compute budget. The main question is whether replacing the standard 1-step critic target with n-step targets improves sample efficiency in continuous-control tasks, and where the horizon becomes unstable. I implement a configurable n-step buffer before the replay buffer and compare SAC with $n=1$, $n=3$, and $n=5$. The experiments

include Pendulum-v1 as a sanity check and HalfCheetah-v5 as the main benchmark. On HalfCheetah-v5, $n=3$ reaches the best final return, while $n=5$ fails to learn a strong running policy under the same hyperparameters. The result suggests that n -step credit assignment can help SAC, but the horizon is a stability parameter rather than a simple bigger-is-better knob.

2 Introduction

Soft Actor-Critic (SAC) is a strong off-policy actor-critic algorithm for continuous-control tasks. In the standard setup, the critic is usually trained with a one-step temporal-difference target. This target is stable and simple, but reward information is propagated backward only one step at a time.

N -step returns propose a more aggressive alternative: accumulate several rewards before bootstrapping. This can improve credit assignment, especially when a useful action affects reward only after several environment steps. However, in off-policy learning this is not free. Longer fragments sampled from the replay buffer may have been generated by older policies, which can increase off-policy bias and target variance.

For this reason, the project does not try to claim a new state-of-the-art algorithm. Instead, it isolates one mechanism - the critic target horizon - and tests it in a small, reproducible experiment.

3 Research question

The main research question is:

Do n -step returns improve SAC under limited compute, and how sensitive is the result to the chosen horizon?

The project compares three variants:

Method	Description
SAC $n=1$	standard 1-step SAC baseline
SAC $n=3$	SAC with a moderate 3-step target
SAC $n=5$	SAC with a longer and riskier 5-step target

The goal is not to prove that one horizon is universally optimal. The goal is to check whether the expected trade-off appears in a compact and reproducible setting.

4 Literature context

The project sits between standard SAC and recent n -step or sequence-aware SAC variants.

- **SAC** is the baseline off-policy actor-critic method with entropy regularization, twin critics, target networks, and a replay buffer.
- **SAC n** is important as a direction that focuses on the difficulty of using n -step returns in off-policy SAC and motivates stabilization or correction mechanisms.
- **T-SAC / Chunking the Critic** is relevant because it uses trajectory chunks and n -step targets together with a sequence-aware critic.

This project does not reproduce full SAC n or T-SAC. Instead, it tests the simpler mechanism shared by these directions: what happens when the SAC critic target horizon changes.

5 Method

The implementation keeps the standard SAC training loop and adds a configurable n-step transition buffer before the replay buffer.

The training pipeline is:

```
environment step
-> n-step buffer
-> replay buffer
-> critic update
-> actor update
-> target network update
-> deterministic evaluation
-> CSV logs and plots
```

For each ready transition, the n-step buffer builds an aggregated transition:

```
state      = s_t
action     = a_t
reward     = r_t + gamma r_{t+1} + ... + gamma^(n-1) r_{t+n-1}
next_state = s_{t+k}
discount   = gamma^k
done       = terminal flag inside the n-step window
```

If an episode terminates before the full window is collected, the window is cut early. For $n=1$, this reduces to standard SAC.

The critic target then becomes:

```
target = reward_n + discount_n * target_value(next_state)
```

This is the only algorithmic component varied across the main ablation. The SAC architecture and hyperparameters are kept fixed for $n=1$, $n=3$, and $n=5$.

6 Experimental setup

The experiments use two continuous-control environments.

Environment	Role	Steps	Seed
Pendulum-v1	sanity check	30,000	0
HalfCheetah-v5	main benchmark	100,000	0

The experiments were run on a Ryzen 5700 laptop through WSL Ubuntu using CPU training. Evaluation uses a deterministic policy and five evaluation episodes per checkpoint. Raw CSV logs and plots are committed to the repository.

Important limitation. These are single-seed experiments. They are useful as bounded reproduction and ablation evidence, but they are not a statistical proof that one horizon is universally best.

7 Results

7.1 Pendulum-v1 sanity check

On Pendulum-v1, all three variants train successfully and reach similar final quality. The main role of this experiment is to verify that the implementation works and that the n-

step target construction does not immediately break SAC on a simple environment.

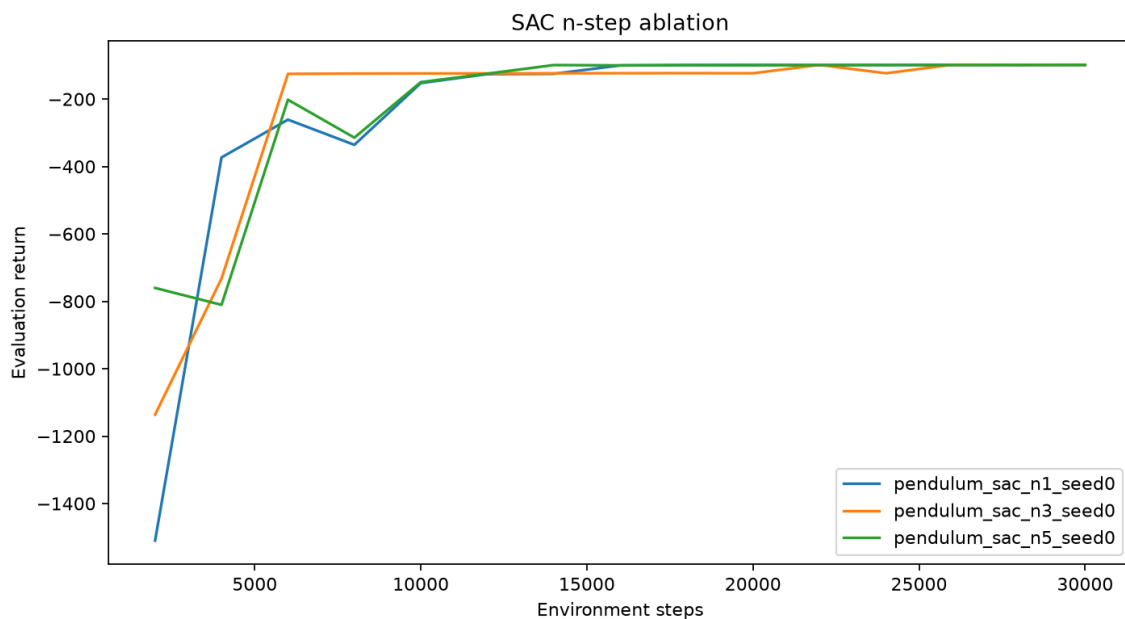


Figure 1. Pendulum-v1 learning curves

The curves suggest that $n=3$ reaches a good return quickly, but the task is too simple to support a strong conclusion about the best horizon. This experiment should be treated as a sanity check, not as the main evidence.

7.2 HalfCheetah-v5 main ablation

The main result is observed on HalfCheetah-v5.

Method	Final eval return	Best eval return
SAC $n=1$	4354.49	4354.49
SAC $n=3$	4578.91	4578.91
SAC $n=5$	764.49	833.37

Main observation. In this single-seed run, $n=3$ gives the best final return and reaches strong performance earlier than the 1-step baseline. The $n=5$ variant fails to learn a strong running policy under the same settings.

The useful part of the result is not only that $n=3$ performs best. The useful part is also the failure of $n=5$: it shows that increasing the n -step horizon is not automatically beneficial in off-policy SAC.

8 Interpretation

The result supports the expected trade-off.

A moderate n -step horizon can improve sample efficiency because reward information is propagated backward faster. This likely explains why $n=3$ improves over the 1-step baseline on HalfCheetah-v5.

At the same time, a longer naive target can increase off-policy bias and target variance. This likely explains why $n=5$ fails under the same hyperparameters.

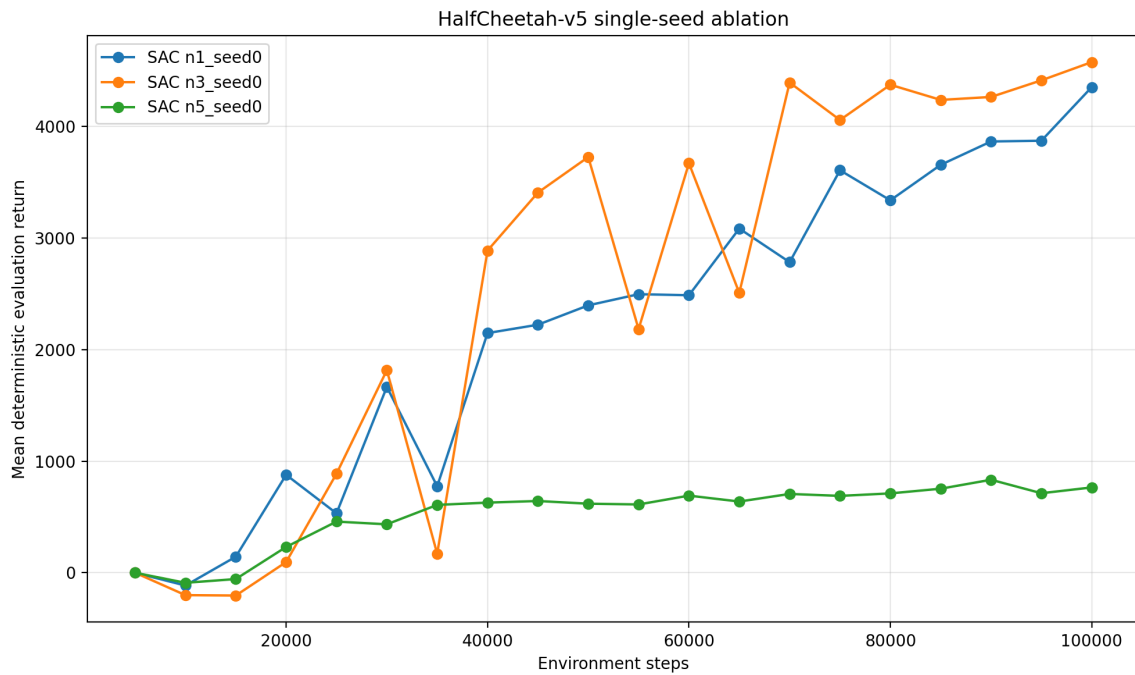


Figure 2. HalfCheetah-v5 learning curves

The safest conclusion is therefore narrow:

In this limited-compute single-seed ablation, 3-step SAC improved HalfCheetah learning compared with the 1-step baseline, while 5-step SAC failed under the same hyperparameters.

The project should not claim that $n=3$ is always best or that $n=5$ always fails. It shows that the horizon is a stability parameter and that naive longer targets can be risky.

9 Limitations

The main limitations are:

- only one random seed;
- only two environments;
- no full SACn correction mechanism;
- no transformer or sequence-aware critic;
- no separate hyperparameter tuning for each horizon;
- limited laptop CPU compute.

These limitations are important. The project is not a paper-scale benchmark and does not make a state-of-the-art claim. Its value is an implementation-level ablation and a clear failure analysis.

10 Future work

The most useful extensions are:

1. run 3-5 seeds;
2. add more MuJoCo environments such as Hopper-v5 or Walker2d-v5;
3. compare naive n -step SAC with SACn-style corrections;
4. tune hyperparameters separately for each horizon;

5. add a sequence-aware critic inspired by T-SAC;
6. report confidence intervals and failure rates.

11 Conclusion

This project implements configurable n-step targets in SAC and tests them in a small but reproducible ablation. Pendulum-v1 confirms that the implementation works. HalfCheetah-v5 shows the main trade-off: $n=3$ helps, $n=1$ remains a strong baseline, and $n=5$ fails under the same hyperparameters.

The final takeaway is:

N-step horizon is a stability parameter, not a bigger-is-better knob.